Brand Review Desk — Project Submission

Project and links

? Source repository: Rudra-Patel-CS/Brand-Review-Desk (https://github.com/Rudra-Patel-CS/Brand-Review-Desk)

? Intended public demo: https://rudra-patel-cs.github.io/Brand-Review-Desk/ (https://rudra-patel-cs.github.io/Brand-Review-Desk/)

? Access: The GitHub Pages build is designed to open directly into a browser-only demo workspace without registration or sign-in.

? Deployment status: The deployment workflow is included, but a live deployment is not confirmed until GitHub Pages is enabled for the repository and a Pages workflow run succeeds. Do not treat the intended URL as live before that check.

What the product does

Brand Review Desk is an existing creative-review application. A reviewer can work with an advertising caption and artwork, compare the content with configured brand rules, inspect the exact rule citation and explanation for each finding, make or queue a correction, dismiss a finding with a written reason, and complete an approval/export flow.

Completed features

1. No-login public demo path: A Vite build flag selects the public demo mode, which bypasses the authentication gate without changing the authenticated server-backed mode. Demo actions do not require a sign-up.
2. Caption checks: The existing rule engine evaluates caption text against enabled brand rules. Findings show the rule identifier/source, matched text, severity, explanation, and suggested correction where configured.
3. Artwork OCR: Tesseract.js reads actual pixels from selected image files in the browser. The app displays extracted text, confidence and available word/line boxes. Users can inspect and correct extracted text; OCR is not represented as a factual or legal verification.
4. Human decision workflow: A reviewer can apply caption suggestions, queue OCR/creative corrections, dismiss a finding with a required note, mark a matter for manual review, re-audit, and approve/export. OCR findings remain unresolved until the replacement artwork is uploaded and checked again.
5. Creative replacement: A reviewer can replace the active campaign's image without creating a separate campaign, rerun OCR and rules, and clear old approval metadata so the new artwork must be reviewed again.
6. Approval and export: The public demo creates a local approval record, keeps it in the current page session, and supports review export. The local demo digest is not a server attestation.
7. Visual guidance: Platform format dimensions, safe-zone overlays, and available OCR-box overlap checks are shown as guidance. The interface explicitly calls for manual visual review; it does not claim to verify contrast, logos, full visual context, or final platform rendering.
8. Existing server-backed capabilities retained: The project still contains the Express/MongoDB application, authenticated sessions, roles, GridFS upload routes, and server-side OCR endpoints. The static judge demo does not use MongoDB.
9. Deployment and handoff: A GitHub Actions Pages workflow, setup instructions, project documentation, and test/build commands are provided in the repository.

Technology and implementation

? Frontend: React 19, TypeScript, Vite 6, Tailwind CSS 4.

? Public demo: Static GitHub Pages build with `VITE_PUBLIC_DEMO_MODE=true`; public data and actions live only in browser memory. Tesseract worker, core, and English language data are copied into `public/vendor` by

`scripts/prepare_ocr_assets.js` during the build.

- ? Server-backed application: Node.js/Express 4, MongoDB 6 driver and GridFS, cookie-based authenticated sessions, and existing role-based workflow/API routes.
- ? OCR: Tesseract.js 7 for browser-side OCR in public demo and the existing server-side OCR route for the authenticated app.
- ? Checks and verification: TypeScript type check, Node test suite, Vite production build, and browser validation. No Supabase was introduced.

How to deploy the no-login demo

1. Open the source repository's Settings → Pages and select GitHub Actions as the source.
2. Push to `main`, or run Actions → Deploy public demo → Run workflow.
3. Confirm the workflow completes successfully and GitHub reports the deployed Pages URL.
4. Open the URL in a new private/incognito browser window with no existing application session. The expected initial state is the sample review workspace, not a login or registration form.
5. Exercise a caption finding, OCR of an uploaded image, a decision, approval, and export.

The Pages workflow does not need application credentials. For a separate server-backed deployment, provide `MONGO_URI`, `DB_NAME`, and authentication secrets through the hosting platform's server-side secret manager. Never put MongoDB credentials in frontend source or any `VITE_*` variable.

Honest scope and unfinished work

- ? Live deployment: Not confirmed until Pages is enabled and a successful deployment is visible in Actions. The intended URL above must not be presented as a verified live link before then.
- ? Public demo persistence: Uploads, OCR corrections, rule edits, findings, decisions, approvals, and exports are in-memory demo behavior and are lost on reload. The static demo has no backend or shared user accounts.
- ? Factual review: Rule matching only checks configured patterns and stored guidance. It does not independently verify claims, substantiation documents, terms, or current facts. A human must verify those against approved evidence.
- ? Visual review: OCR-box/safe-zone checks are limited to available OCR output and configured platform geometry. Contrast, logo placement, image context, and platform-specific rendering still require a human.
- ? Backend hosting: The existing MongoDB/Express mode remains available in source, but it is not the public no-login Pages demo and needs a separately configured server and database.
- ? Legal effect: A browser-only approval record or local hash is not a legal certification, secure signature, or immutable audit record.

Changes from the earlier plan

- ? Preserved and improved the existing React/TypeScript application rather than rebuilding it. Kept its Express/MongoDB server-backed mode and added a separate static, no-login Pages demo to satisfy judge access.
- ? Used the existing transparent deterministic rules engine instead of introducing an LLM or external fact-checking service. This keeps findings tied to configured rule citations and avoids presenting generated text as verification.
- ? Kept browser demo uploads and decisions in memory rather than adding a public database or exposing credentials. Persistent multi-user review remains the responsibility of the existing authenticated backend deployment.
- ? Clarified that OCR and safe-zone analysis are limited automation, not visual/legal certification; added a true replacement-artwork path for image findings instead of marking them resolved by editing only the caption.

Three meaningful Round 3 improvements

1. Evidence-backed claim verification: Add a controlled evidence library for approved claims, terms, expiration dates, and source documents. Require each factual finding/approval to link to a current evidence record; flag missing or expired evidence for a human rather than inferring truth from wording.
2. Shared, durable review operations: Deploy the existing API and MongoDB with organization workspaces, durable campaign/rule/audit records, reviewer assignments, and a tested backup/restore process. Keep credentials server-side and retain role separation and self-approval controls.
3. Validated visual QA pipeline: Add reproducible image analysis for contrast, logo presence/clear-space, and text visibility against target-format overlays, with measured confidence and cited configuration. Low-confidence or unsupported cases must stay in a human-review queue; validate results against a labeled creative test set before enabling any automated pass.

Verification record

? `npx tsc --noEmit` — passed.

? `npm test` — passed (63 tests, including the browser download helper regression test).

? Public-demo production build with `VITE_BASE_PATH=/Brand-Review-Desk/` and `VITE_PUBLIC_DEMO_MODE=true` — passed.

? Browser/incognito verification and the public Pages deployment must be recorded only after the current build has been exercised in a clean browser profile and a successful Pages deployment is available.